1. Introducción y uso básico del sistema

1.2. Uso de CLI

Felipe Ortega, Daniela Feversani, Elena García-Morato

Laboratorio de Sistemas (LSIS)
GSyC, EIF. URJC

2026

GSyC

Universidad Rey Juan Carlos

Licencia



(cc) 2025-2026 Felipe Ortega, Daniela Feversani, Elena García-Morato
Algunos derechos reservados. Este trabajo se entrega bajo la licencia Creative Commons Reconocimiento - NoComercial - SinObraDerivada (by-nc-nd). Para obtener la licencia completa, véase https://creativecommons.org/licenses/by-nc-nd/3.0/es/.

Tema 1 - Introducción y uso básico del sistema

1.2. Uso básico de Command Line Interface

Contenidos

- 1.1 Introducción a la CLI
- 1.2 Gestión de procesos
- 1.3 Variables y aritmética
- 1.4 Expansión de nombres de ficheros
- 1.5 Redirecciones
- 1.6 Listas de órdenes
- 1.7 Tuberías (*pipes*)

1.1 Introducción a la CLI

Command Line Interface

- Tal y como se explicó en el tema anterior, la interfaz gráfica de usuario (**GUI**) se utiliza habitualmente para navegar y realizar tareas de forma sencilla, pero resulta ineficiente para las tareas de administración del sistema.

- Para la realización de tareas más específicas o complejas utilizaremos la interfaz de linea de comandos (**CLI**, Command-Line Interface), un programa que nos permite interactuar con el sistema operativo usando solo teclado y pantalla en modo texto.
  - El usuario escribe directamente las órdenes a través de un **prompt**, de tal manera que son interpretadas y ejecutadas inmediatamente por el sistema operativo.
  - El conjunto de comandos internos y elementos disponibles, así como su sintaxis, dependerá de la shell utilizada.

Funciones

Además de la ejecución de comandos, la shell también es compatible con las siguientes funciones:

- Historial de comandos, que permite acceder a los comandos ingresados anteriormente
- Administración de variables de entornos
- Estructuras de control y funciones
- Redirección de la entrada y la salida
- Pipes, que permiten conectar la salida de un comando con la entrada de otro

Funcionamiento de la CLI

La **CLI** implementa un bucle que permite ejecutar órdenes de manera inmediata de acuerdo a la siguiente secuencia:

- Se muestra el *prompt*
- El usuario **introduce** el comando que desea ejecutar
- Se **interpreta** el comando introducido comando para comprender su estructura e identificar la orden lanzada, las opciones y los argumentos
- El sistema operativo **ejecuta** el comando
- En caso de que el comando genere elementos de **salida** se muestran por pantalla
- Se muestra de nuevo el *prompt* y se espera a recibir otro comando.

1.2 Gestión de procesos

Foreground vs. *background*

- Normalmente, las órdenes externas se ejecutan de forma interactiva (***foreground***): la shell interpreta la orden, la ejecuta, nos muestra la salida de la orden, y no nos devuelve el *prompt* hasta que no acaba de ejecutarse el proceso.
- No obstante, también existe la posibilidad de dejar un programa en ejecución de manera no interactiva o en segundo plano (***background***).

Ejecución de un proceso en *background*

- Para **lanzar un proceso en segundo plano** se emplea el operador **&**.
- Por ejemplo, el programa `xlogo` se lanzaría en segundo plano a través del siguiente comando:

```
$ xlogo &
[2] 7584
$
```

La salida del programa nos devuelve dos datos:

- El primer número, en este caso el [2], es el **número de tarea** (*job*)
- El segundo número (7584) es el **identificador de proceso**, el **PID**.

Foreground vs. Background

- La orden **`bg`** (de *background*) nos permite pasar a segundo plano un proceso que se encuentra suspendido, por ejemplo porque se le ha enviado la señal correspondiente mediante la combinación de teclas Ctrl+Z.
- La orden **`fg[tid]`** (de *foreground*) permite traer a primer plano (*foreground*) un proceso que se encontraba en *background*.

Procesos en ejecución

- Para consultar cuáles son los **procesos** que se están **ejecutando actualmente en segundo plano** puede emplearse la orden **`jobs`**.

Finalización de procesos

- Una vez que el proceso ha iniciado la ejecución como un proceso separado, ya no podemos interrumpirlo mediante la combinación Ctrl-C.
- Para terminar la ejecución de un proceso en *background* hemos de enviarle la señal de terminación (SIGTERM) mediante la orden **`kill`**, indicando como parámetro bien el número de PID, bien el número de tarea precedido por el carácter %.
- Si no responde a SIGTERM, podemos enviar SIGKILL.
- Cuando un proceso que se estaba ejecutando en *background* finaliza, la shell muestra un mensaje indicando tal circunstancia.

Ejercicio

Ejercicio:

- Lanza 4 procesos `xeyes` en *background*
- Trae a *foreground* y a continuación detén los 2 procesos creados en segundo y cuarto lugar
- Termina los 2 procesos que siguen en ejecución usando la orden `kill`
- Trae los 2 restantes a *foreground* (uno a uno) y elimínalos con `Ctrl-C`

1.3 Variables y aritmética

Definición de variables

- *Bash* permite la **definición de variables** cuyo valor puede ser recuperado más tarde.
- Por ejemplo:

```
$ edad=24
$ echo $edad
24
$ saludo=”Hola, cómo estás?”
$ echo $saludo
”Hola, cómo estás?”
$
```

Definición de variables

- Al asignar un valor a una variable no deben aparecer espacios alrededor del símbolo $=$.
- Resulta un error en caso contrario, pues la shell considera que el nombre de la variable es una orden que debe ejecutar con una serie de argumentos
- Por ejemplo:

```
$ edad = 24
bash: edad: command not found
$
```

Operaciones aritméticas

- Se pueden realizar operaciones aritméticas sobre variables mediante el operador `$(([expresión]))`.
- Pueden usarse operadores aritméticos y de bits: $+, -, /, *, \%, \&, |, \wedge, <<, >>$
- Por ejemplo:

```
$ edad=24
$ echo $edad
24
$ edad=$(( $edad+1 ))
$ echo $edad
25
$
```

Variables de entorno

- Podemos convertir una variable en **variable de entorno** mediante la orden **`export`**.
- Las variables de *shell* están disponibles sólo para la *shell* en sí, mientras que as variables de entorno están disponibles para cualquier programa lanzado desde la *shell*.
- Podemos ver las variables de entorno con la orden `printenv`

Ejercicio

Ejercicio:

- Crea una variable con nombre a y valor 5
- Crea una variable b con valor a+3
- Crea una variable c con valor b desplazado 3 bits a la izquierda.
- Imprime por pantalla los valores de a, b y c

Sustitución de órdenes

- Mediante la **sustitución de órdenes** también podemos almacenar en una variable la salida estándar de una orden.
- Para ello, se utiliza la sigiente fórmula : `$([orden])`
- Por ejemplo:

```
$ nombre=$( whoami )
$ echo $whoami
elenagm
$ n_archivos=$( ls | wc -w )
$ echo $n_archivos
4
$
```

1.4 Expansión de nombres de ficheros

Expansión de nombres de ficheros

- A veces queremos referirnos a un conjunto de ficheros que tienen características comunes en sus nombres.
- Por ejemplo, querremos seleccionar todos los archivos con una extensión determinada, que empiecen por una letra, o cuyo nombre contenga una cadena de caracteres en concreto.

Comodines

- Para tal propósito nos valdremos de la función de **expansión de nombres de ficheros** de la shell, que antes de ejecutar un comando escanea cada cadena de caracteres proporcionada como argumento en busca de caracteres **comodín** '*', '?' y '['.
- En caso de encontrar alguno de ellos, la palabra es considerada como un patrón, y reemplazada por una lista alfabéticamente ordenada de nombres de ficheros que se ajusten a dicho patrón.
  - El carácter `*` concuerda con cualquier cadena de caracteres.
  - El carácter `?` concuerda con cualquier carácter.
  - `[conjunto]` concuerda con cualquier carácter dentro de `conjunto`.

Comodines

- Ejemplo: Si queremos mostrar en pantalla (usando el comando `cat`) todos los ficheros que contengan código fuente C, la característica común es que los nombres de los ficheros acaban en .c, por lo que utilizaríamos el siguiente comando:

```
$ cat *.c
```

- Ejercicio: ¿Qué ficheros listaría la orden `ls ?[a-c]*.txt` ?

1.5 Redirecciones

Descriptores de ficheros

- Para facilitar el acceso a los distintos recursos del sistema (zona de almacenamiento en disco, teclado, pantalla, socket, ...), el kernel de Linux vincula cada recurso con un fichero, real (en disco) o virtual (en memoria).
- Un **descriptor de fichero** es un número entero que identifica de manera única al fichero asociado a un proceso.
- Todo **proceso** Unix tiene asociados de manera predeterminada tres descriptores de ficheros: la entrada estándar (0), la salida estándar (1) y la salida de error (2).
- A la salida de error se dirigen los **mensajes provocados por algún error** en la ejecución del proceso.

Redirección de la salida estándar

- La salida estándar de una orden puede **redirigirse a un fichero** mediante el operador **`>[fichero]`**. Por ejemplo: `$ ls -l /etc > dir_etc.txt`
- Con el operador `>[fichero]` se trunca el fichero designado. Si queremos añadir el resultado de una orden al contenido de un fichero existente, debemos utilizar el operador de doble redirección » `[fichero]`

Redirección de la salida de error

- La salida de error de una orden también puede redirigirse a un fichero mediante el operador `2>[fichero]`. A través de esta orden, lo que se hace es básicamente **redirigir la salida de error** indicando el número de descriptor de fichero (2) justo antes del operador de redirección.

- A través del uso del operador `>&[fichero]` también podemos **redirigir la salida de error a la salida estándar**, para que al redirigir esta última a un fichero aparezcan redirigidos tanto los mensajes de error como la salida normal.

Ejemplo

```
$ ls -l /bin/bash /bin/noexiste
ls: no se puede acceder a /bin/noexiste: No existe el fichero o directorio # Salida de error
-rwxr-xr-x 1 root root 700492 may 12 2008 /bin/bash # salida estándar

$ ls -l /bin/bash /bin/noexiste >/dev/null  # Se redirige la salida estándar
ls: no se puede acceder a /bin/noexiste: No existe el fichero o directorio # Salida de error

$ ls -l /bin/bash /bin/noexiste 2>/dev/null  # Se redirige la salida de error
-rwxr-xr-x 1 root root 700492 may 12 2008 /bin/bash # salida estándar
```

- Ejercicio: Repite el ejemplo anterior, pero volcando tanto la salida estándar como la de error a un fichero llamado `salidas.txt`

1.6 Listas de órdenes

Listas de órdenes

Una lista de órdenes es una **secuencia de comandos** (con redirección o no) separados por los operadores: `;` `&` `&&` `||`

- En una lista de órdenes separadas por el operador ; la shell ejecuta las órdenes de manera secuencial, esperando que se complete cada uno de los comandos. El **código de retorno** (el valor de la variable **`$?`**) es el de la última orden ejecutada.

Listas de órdenes

- Si el operador es `&`, el comando se ejecuta en segundo plano, la shell no espera a que termine y el código de retorno es 0.
- Los operadores `&&` y `||` son operadores AND y OR respectivamente.
  - `orden1 && orden2` → solo se ejecuta orden2 si orden1 retornó con código 0
  - `orden1 || orden2` → solo se ejecuta orden2 si orden1 retornó con código distinto de 0

Ejercicio

Ejercicio: ¿En cuales de los siguientes casos sería 0 el código de retorno?

- `ls /bin/bash && echo Existe ; echo $?`
- `ls /bin/noexiste && echo existe ; echo $?`
- `ls /bin/bash || echo No existe ; echo $?`
- `ls /bin/noexiste || echo No existe ; echo $?`

1.7 Tuberías (*pipes*)

Tuberías (*pipes*)

- Una tubería (comúnmente llamada ***pipe***) es una secuencia de uno o más comandos (simples o compuestos, pero no ningún tipo de lista) **separados por el operador |**.
- En un *pipe*, la salida estándar de un comando se conecta a la entrada estándar del siguiente comando.
- Por ejemplo:

```
elenagm@f-l3109-pc23:~$ ls -l /etc | sort | more
drwx------  2 root                root                4096 ago  8 16:51 credstore
drwx------  2 root                root                4096 ago  8 16:51 credstore.encrypted
drwx------  2 root                root                4096 ene 20 13:06 iptables
drwx------  2 root                root                4096 sep  2 17:19 multipath
drwx------  2 root                root                4096 sep  2 18:32 gitlab-runner
drwxr-s---  2 root                dip                 4096 sep  2 16:39 chatscripts
drwxrwxr-x  2 root                landscape           4096 ago 27 16:26 landscape
```

Ejercicio

Ejercicio:

- Encadenando varias órdenes a través de una sola tubería, imprime por pantalla el número de archivos que contiene el directorio `/Escritorio`.

pipes y redirecciones

- **Se pueden combinar** *pipes* y redirecciones.
- Es muy importante tener en cuenta que las redirecciones de salida se realizan antes de ejecutar las órdenes.

Ejercicio

Ejercicios:

- Encadenando varias órdenes a través de una sola tubería, escribe en un fichero llamado `alumnos.txt` el login de todos los usuarios del directorio `/home/alumnos`.
- Encadenando varias órdenes a través de una sola tubería y leyendo el fichero anterior, escribe en la salida estándar el número de alumnos cuyo login empieza por la letra `z`